



Advanced NLP

Session 3 - Neural Network Basics & Backpropagation in NLP

An intuitive, diagram-first walkthrough

Teacher: Dr. Behrouz Minaei-Bidgoli

Presenter: Alireza Nazari

Presentation creator: Alireza Nazari

Contents

1. Neural Networks: The Building Blocks

neurons, layers, embeddings, non-linearities

2. Training: Learning from Mistakes

the training loop, loss, gradient descent, learning rate

3. Backpropagation: How Gradients Flow

computation graphs, one node's job, a worked example, autodiff in PyTorch

4. In Practice: Making Training Work

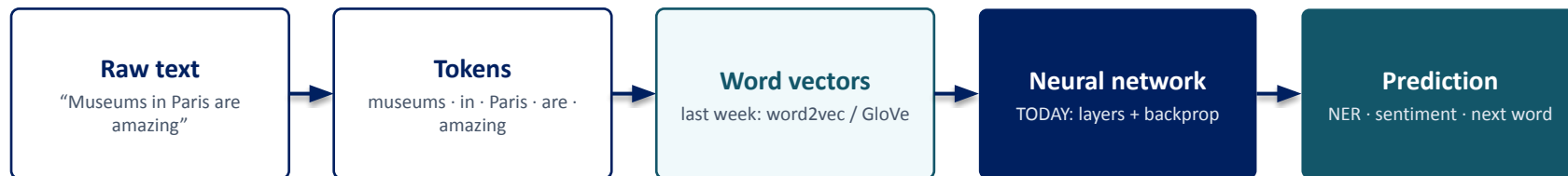
overfitting, dropout, initialization, optimizers, a tiny PyTorch classifier

1.

Neural Network Building Blocks

Based on Stanford CS224N Lecture 3 (Yang, 2025) & Lecture Notes Part III (2019)

Where Neural Networks Fit in the NLP Pipeline



TODAY'S THREE QUESTIONS

1. What is a neural network, concretely?
2. How does it learn from its mistakes?
3. How are millions of gradients computed so fast? → backpropagation

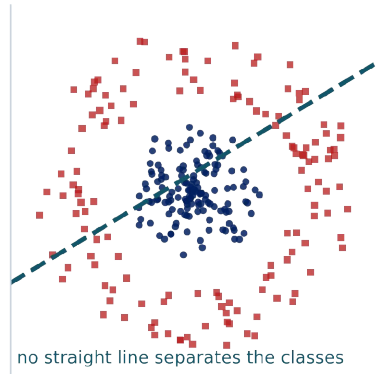
WHY THIS WEEK MATTERS

The one week to really work through the material

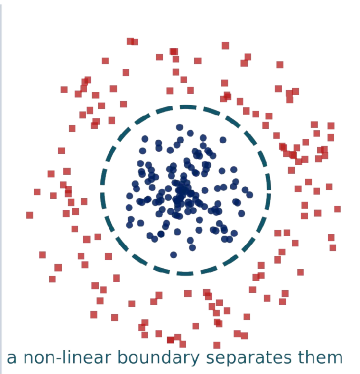
Every later model – RNNs, Transformers, LLMs – is trained exactly this way. Frameworks hide the math, but debugging them requires knowing what happens underneath.

Why Not Just a Linear Classifier?

Linear classifier (softmax / logistic)



Neural network



Same data, two classifiers. The dashed line is the learned decision boundary.

LINEAR MODELS

Draw one straight line

Softmax / logistic regression: $\text{score} = W \cdot x$.
Great when classes are separable by a hyperplane – language rarely is.

NEURAL NETWORKS

Bend the boundary

Stack simple units with non-linearities between them → any curved boundary, learned from data.

From a Biological Neuron to an Artificial One

THE ANALOGY

A neuron: many inputs in, one number out

Dendrites → inputs $x_1, x_2, \dots$ the word-vector entries

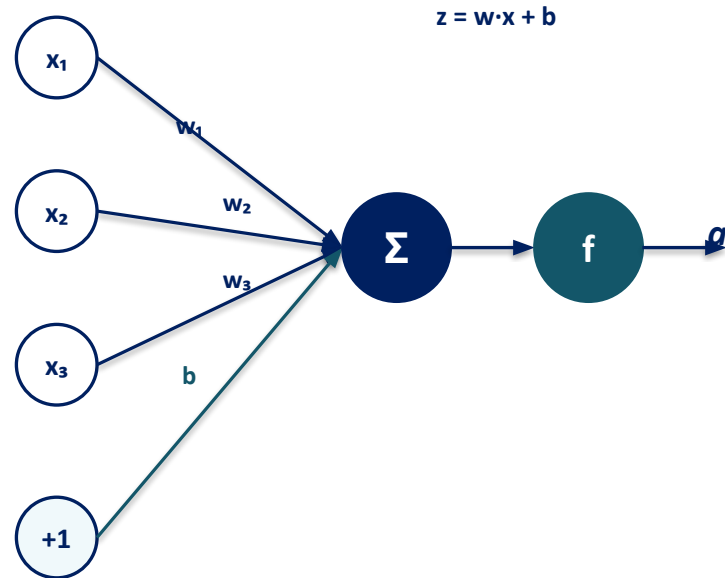
Synapse strength → weights w : how much each input matters

Cell body → weighted sum $z = w \cdot x + b$

Firing → activation $a = f(z)$, a squashing function

Different neurons differ only in their weights

Don't over-stretch the biology: artificial neurons are a tiny caricature of real ones.

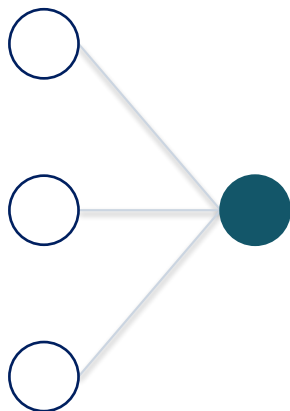


$$a = f(w^T x + b)$$

the only equation you need for a neuron

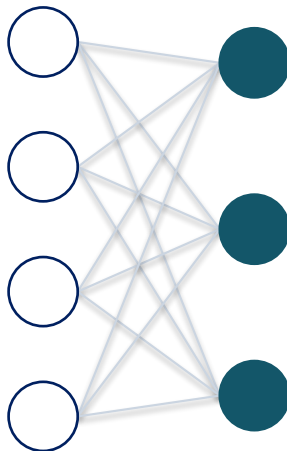
Stacking Neurons: Neuron $\rightarrow$ Layer $\rightarrow$ Network

inputs (white) $\rightarrow$ hidden neurons (navy) $\rightarrow$ output (teal). A layer is just $W \cdot x + b$ followed by f , applied to all neurons at once.



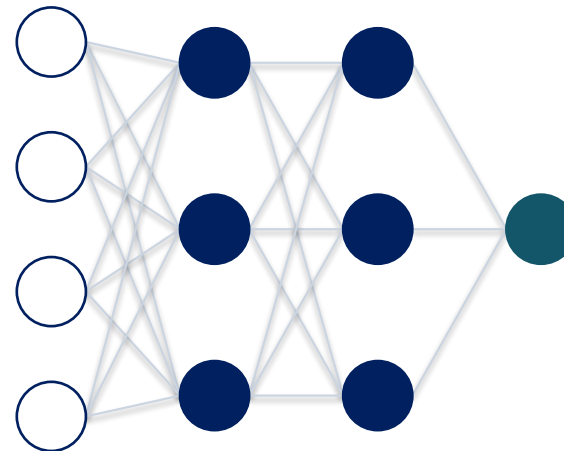
one neuron

3 inputs $\rightarrow$ 1 output



a layer

every neuron sees the same input



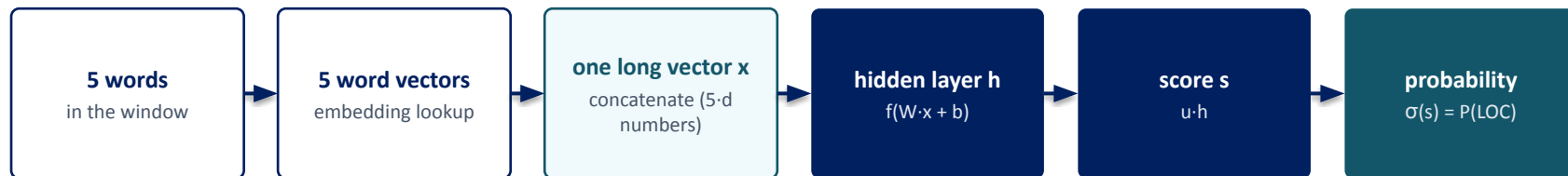
a deep network

each layer re-represents its input

Our Running Example: Is “Paris” a Location?



window classification: look at the center word plus 2 neighbours on each side



WHY A HIDDEN LAYER?

It can learn interactions

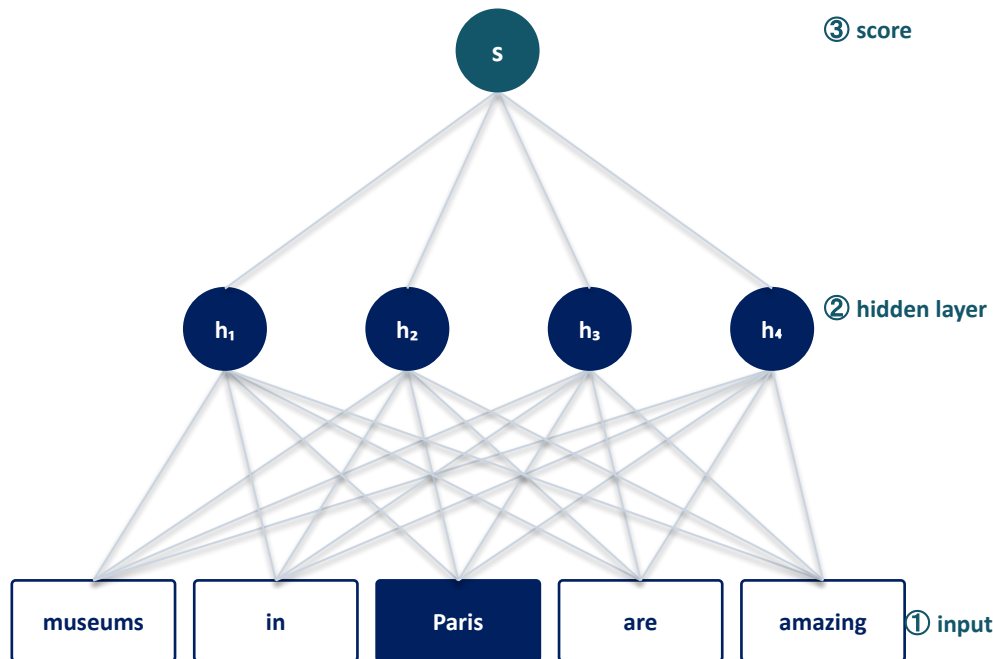
Maybe “museums” only matters when “in” comes right after it. A linear model on the raw vector cannot express that; a hidden layer can.

OUTPUT

One number per window

Slide the window along the sentence and repeat – one score per word. (Real systems use a softmax over many entity types.)

Inside the Network: The Forward Pass, Step by Step



the same network, drawn bottom-up

① INPUT

Concatenate the 5 word vectors

x has 5-d numbers (e.g. $d = 4 \rightarrow 20$ numbers).

$$x \in \mathbb{R}^{5d}$$

② HIDDEN LAYER

Weighted sums, then squash

W has 4 rows (one per hidden neuron) $\times 20$ columns; f is applied to each of the 4 numbers. $h = f(Wx + b)$

③ SCORE

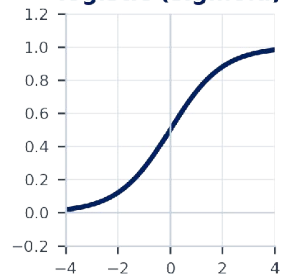
One more weighted sum

u has 4 weights; the dot product gives a single score s . $\sigma(s)$ turns it into a probability.

$$s = u^T h$$

Non-Linearities, Old and New

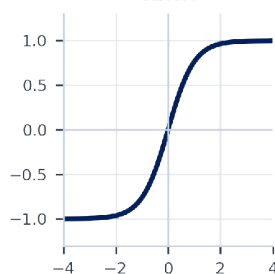
logistic (sigmoid)



$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\partial: \sigma' = \sigma(1-\sigma)$$

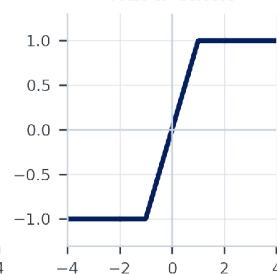
tanh



$$\tanh(z) = 2\sigma(2z) - 1$$

$$\partial: 1 - \tanh^2(z)$$

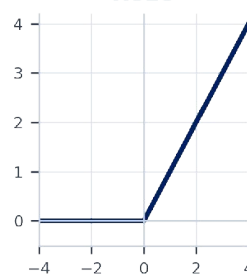
hard tanh



$$\text{clip}(z, -1, 1)$$

$$\partial: 1 \text{ if } |z| \leq 1 \text{ else } 0$$

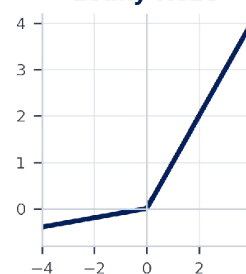
ReLU



$$\max(z, 0)$$

$$\partial: 1 \text{ if } z > 0 \text{ else } 0$$

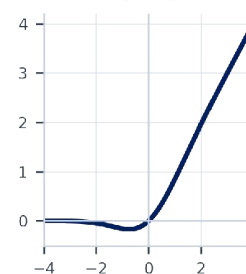
Leaky ReLU



$$\max(z, kz), \quad 0 < k < 1$$

$$\partial: 1 \text{ if } z > 0 \text{ else } k$$

GELU



$$z \cdot P(X \leq z), \quad X \sim N(0, 1)$$

$$\partial: \approx z \cdot \sigma(1.702 z)$$

STILL USED

Logistic & tanh

e.g. logistic to get a probability; tanh is just a rescaled, shifted sigmoid (2x as steep, range $[-1, 1]$)

FIRST THING TO TRY

ReLU

Trains quickly and performs well thanks to good gradient back-flow. Has a negative “dead zone” that Leaky / Parametric ReLU mitigate.

TRANSFORMERS

GELU / Swish

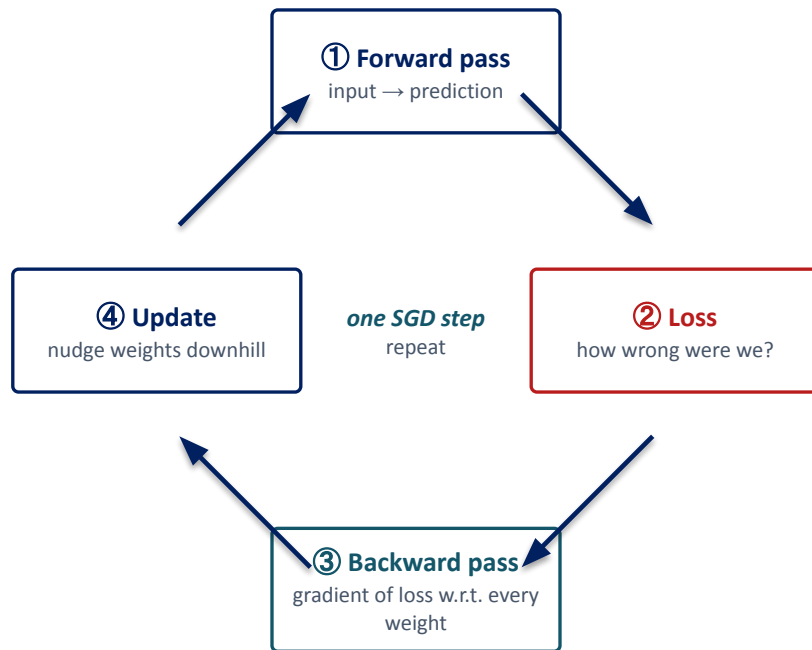
GELU is frequently used with Transformers (BERT, RoBERTa, ...). Swish: $x \cdot \text{logistic}(x)$.

2.

Training: Learning from Mistakes

How a network turns wrong answers into better weights

The Training Loop: Four Steps, Repeated Thousands of Times



① FORWARD

Run the network on a mini-batch; save every intermediate value (we will need them in ③).

② LOSS

A single number measuring the error, e.g. cross-entropy. Lower is better.

③ BACKWARD

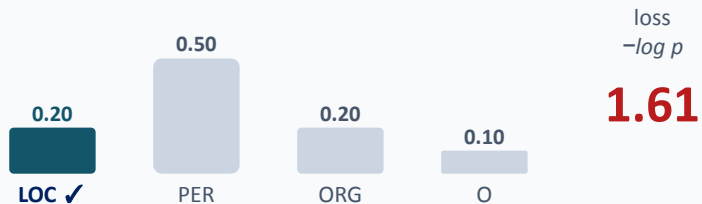
Backpropagation: how much would the loss change if each weight moved a little? That's the gradient.

④ UPDATE

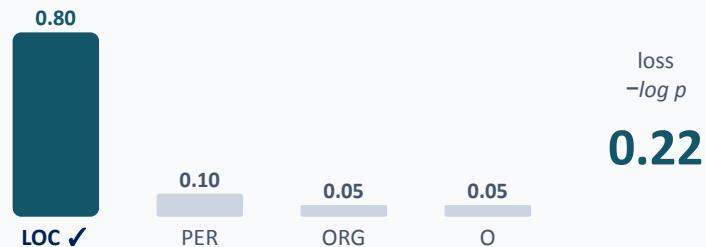
$\theta \leftarrow \theta - \alpha \cdot \text{gradient}$: a small step for every parameter – including the word vectors.

Measuring Mistakes

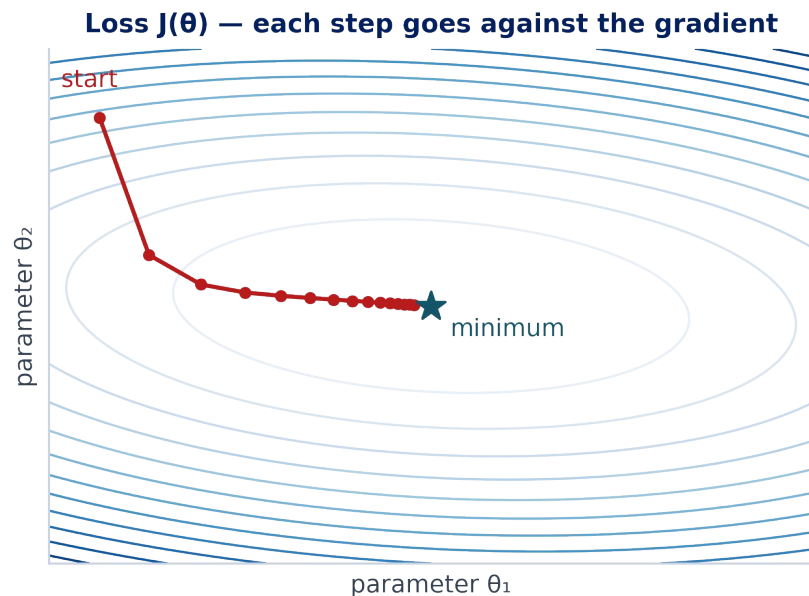
BEFORE TRAINING – model's probabilities for “Paris”



AFTER TRAINING



Gradient Descent: Rolling Downhill on the Loss Surface



θ = all parameters: W , b , u — and the word vectors L

THE GRADIENT

Points uphill — so step the other way

For each parameter: “if I increase it a bit, does the loss go up or down, and how fast?”

THE UPDATE RULE (the only formula here)

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} J(\theta)$$

STOCHASTIC

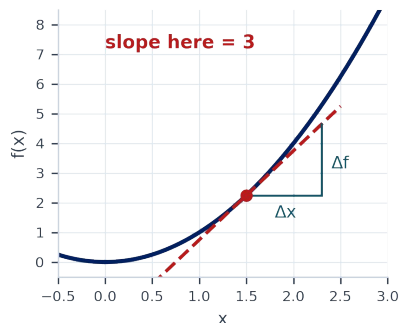
Use a mini-batch, not the whole dataset

Noisier arrows, but thousands of cheap steps beat a few perfect ones.

Derivative → Partial Derivative → Gradient

① DERIVATIVE

One knob



You have one input x and one output f . **Turn the knob a tiny bit — how much does the output move?**

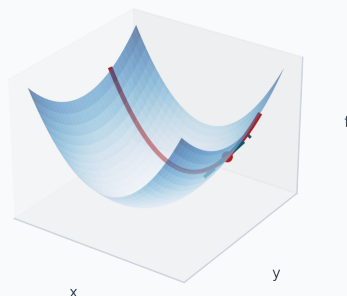
That ratio is the **slope** of the curve at that point. Here the slope is 3: nudge x by 0.01 and f moves by about 0.03.

Written df/dx

② PARTIAL DERIVATIVE

Many knobs, turn one at a time

hold y fixed → slice → ordinary slope



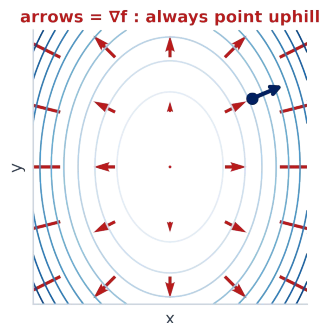
Now f depends on two inputs, x and y , so it is a surface. **Hold y fixed, turn only x .**

Holding y fixed cuts a **slice** through the surface (red curve). That slice is an ordinary derivative. Same for y .

Written $\partial f/\partial x$ and $\partial f/\partial y$

③ GRADIENT

All knobs at once



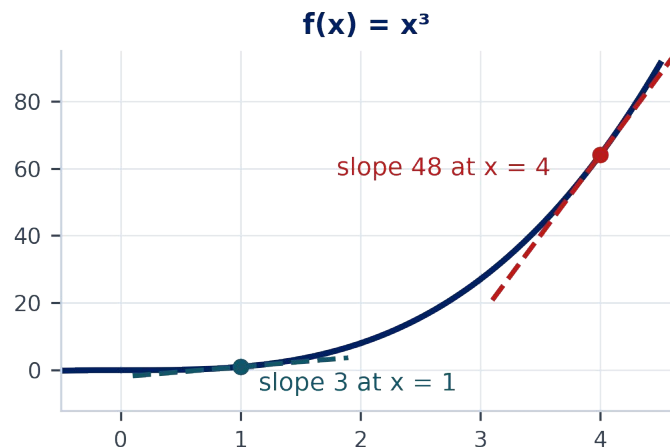
Put every partial derivative into one list: $[\partial f/\partial x, \partial f/\partial y]$. That list is the gradient.

Read as an arrow, it **points uphill** — the direction that makes f grow fastest. Walk against it and f shrinks fastest.

Written ∇f

How they connect: a derivative answers “how sensitive is f to this one input?”. A partial derivative asks the same for one input among many. The gradient collects all those answers. **In a neural network the inputs are the weights and f is the loss — so the gradient tells every weight which way to move.**

What Is a Gradient, Intuitively?



ONE KNOB

Slope = sensitivity

"If I turn this knob by a tiny amount, how much does the output move?"
At $x = 1$ the output moves $3\times$; at $x = 4$ it moves $48\times$.

MILLIONS OF KNOBS

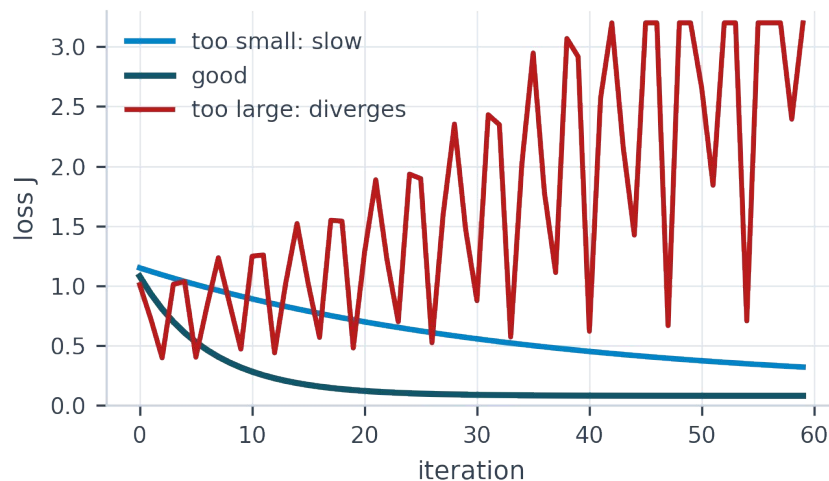
One slope per parameter

The gradient is the list of all those slopes – for W , b , u and every word vector.

THE CHALLENGE

Measuring each slope by nudging one knob and re-running the network would take one forward pass per parameter – millions of passes per step. **Backpropagation gets all of them in one backward pass.**

The Learning Rate: Too Big, Too Small, Just Right



α is the single most important hyper-parameter to tune

TOO LARGE

Overshoots and diverges

Steps jump across the valley; the loss oscillates or explodes.

TOO SMALL

Crawls

Safe but slow; may stall in a shallow region.

IN PRACTICE

Start higher, then anneal

Decay α over time (step, exponential or $1/t$ schedules) – fast at first, careful near the minimum. Adaptive optimizers (Adam) do much of this for you.

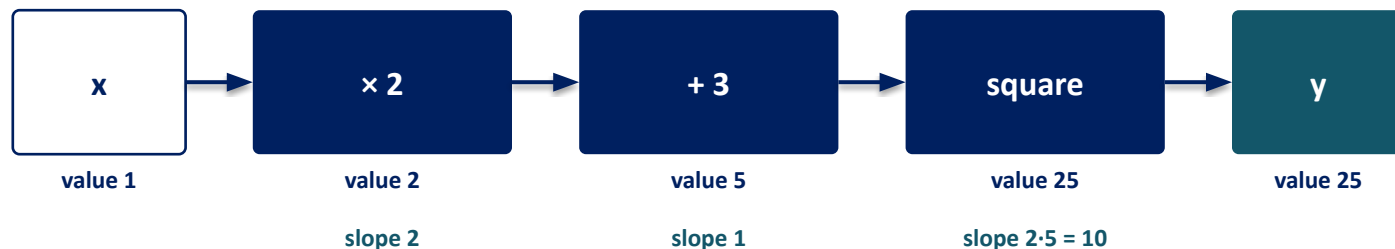
3.

Backpropagation: Gradient Flow

One idea – downstream gradient = upstream gradient $\times$ local gradient – applied node by node

The Chain Rule, Without the Algebra

Nudge the input a little – how much does the output move? Multiply the local slopes along the path.



FORWARD: $x = 1 \rightarrow y = 25$

BACKWARD: total slope = $2 \times 1 \times 10 = 20$

Check: $x = 1.01 \rightarrow (2.02 + 3)^2 = 25.2004 \approx 25 + 20 \times 0.01$ ✓

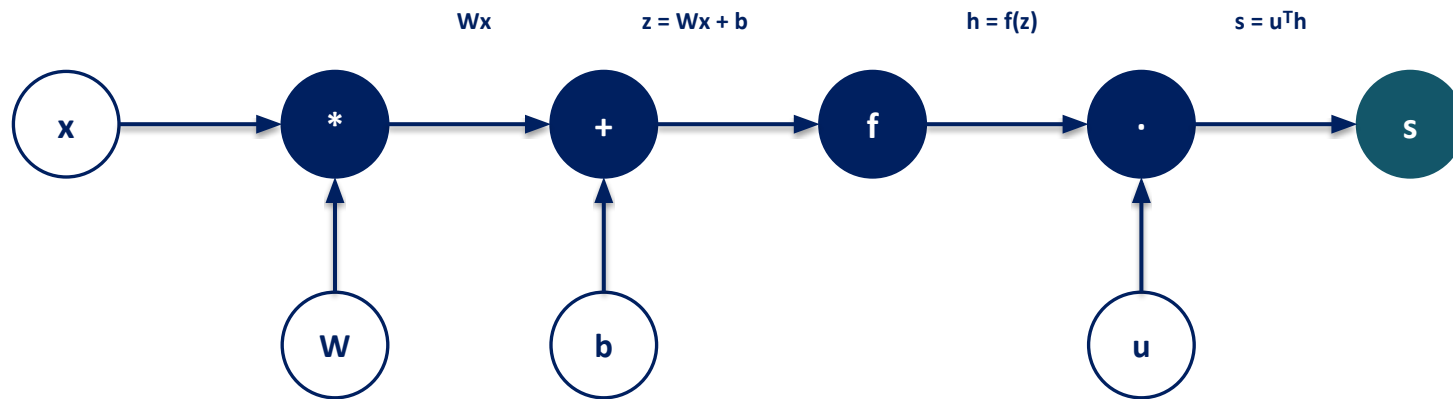
WHY THIS SCALES

Each box only needs its own local slope.

Nobody needs to understand the whole pipeline – walk backwards, multiplying as you go. That is backpropagation.

Computation Graphs: The Network as a Wiring Diagram

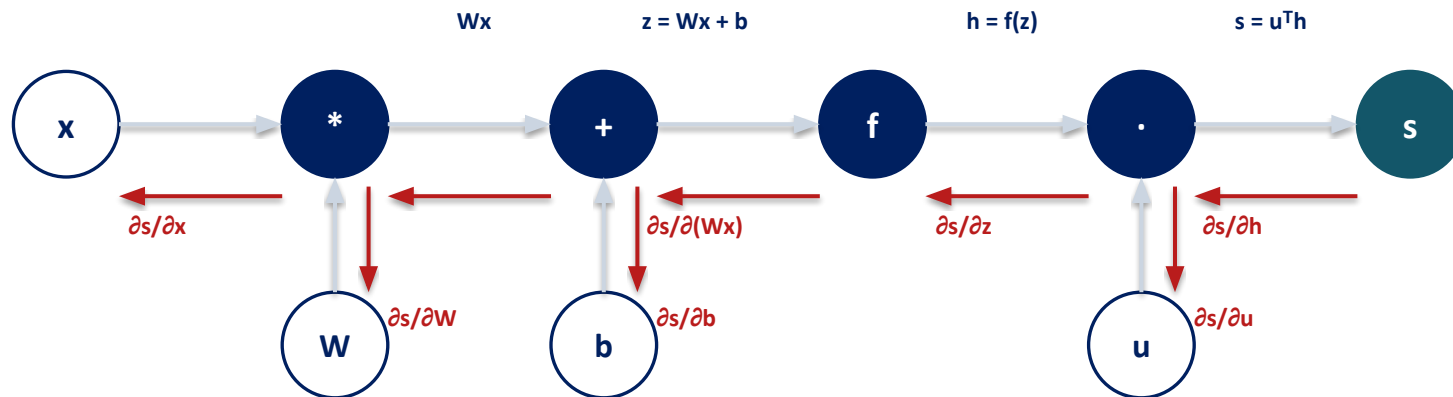
- **Frameworks represent the network as a graph** – nodes are operations, edges carry values
 - Inputs and parameters (x , W , b , u) are the source nodes
 - Every intermediate result (Wx , z , h) is saved during the forward pass
- **Forward propagation** = evaluate nodes left to right



$s = u \cdot f(Wx + b)$ drawn as a graph – values flow left $\rightarrow$ right

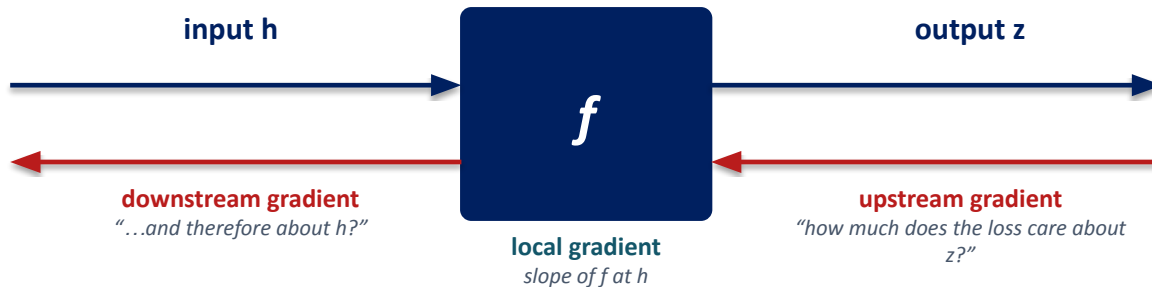
Backpropagation: Walk the Same Graph Backwards

- **Start at the output** with gradient 1 and move right $\rightarrow$ left
 - Each node multiplies the gradient it receives by its own local slope and hands it to its inputs
 - When the walk reaches W , b or u , that is the gradient SGD needs
- **Every edge is visited once** – all gradients in a single pass



red = the backward pass; each red label is a gradient of s

One Node's Job



$$\text{downstream gradient} = \text{upstream gradient} \times \text{local gradient}$$

RECEIVES

one number from the right

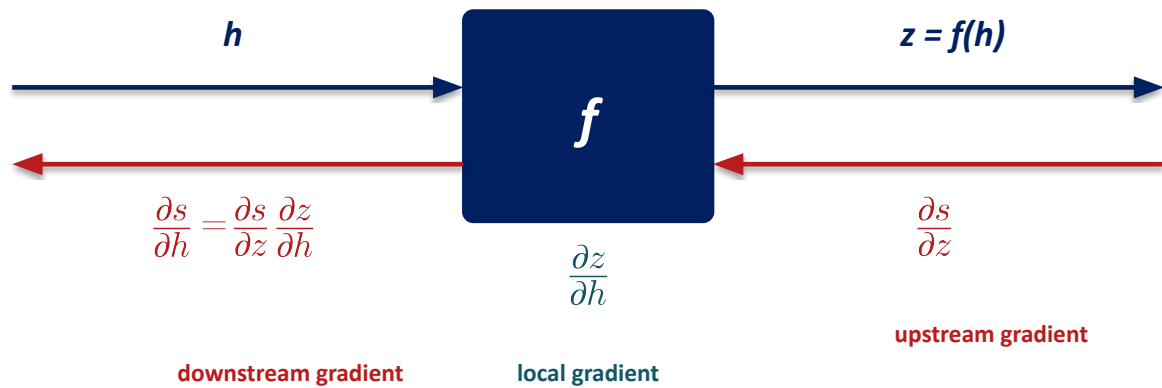
KNOWS

its own slope (e.g. ReLU: 1 or 0)

PASSES LEFT

one number per input

Backpropagation: A Single Node



RECEIVES

an upstream gradient

$\partial s / \partial z$ – how the final output changes with this node's output

KNOWS

its local gradient

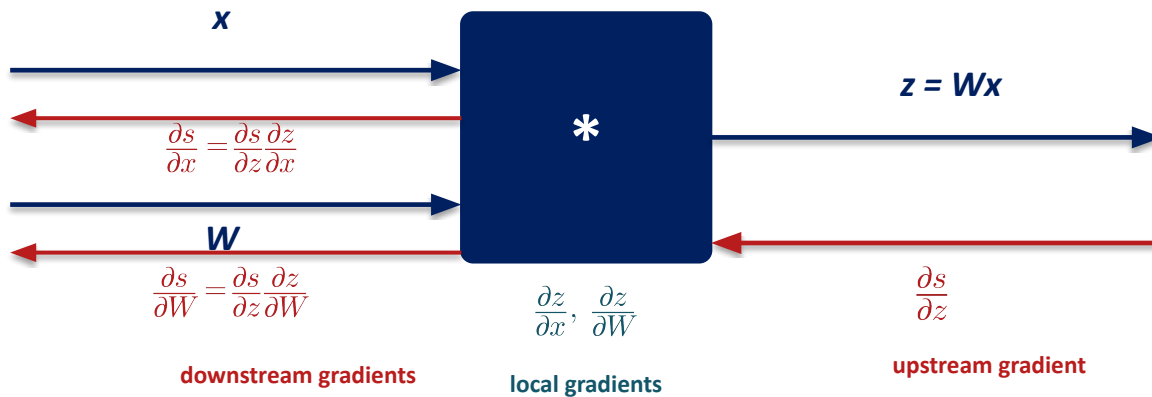
$\partial z / \partial h$ – the derivative of its own output w.r.t. its input

PASSES ON

the downstream gradient

$[\text{downstream}] = [\text{upstream}] \times [\text{local}]$ – that's the chain rule!

A Single Node with Multiple Inputs



MULTIPLE INPUTS

→ **multiple local gradients**

One local gradient per input; each downstream gradient = the same upstream gradient $\times$ its own local gradient.

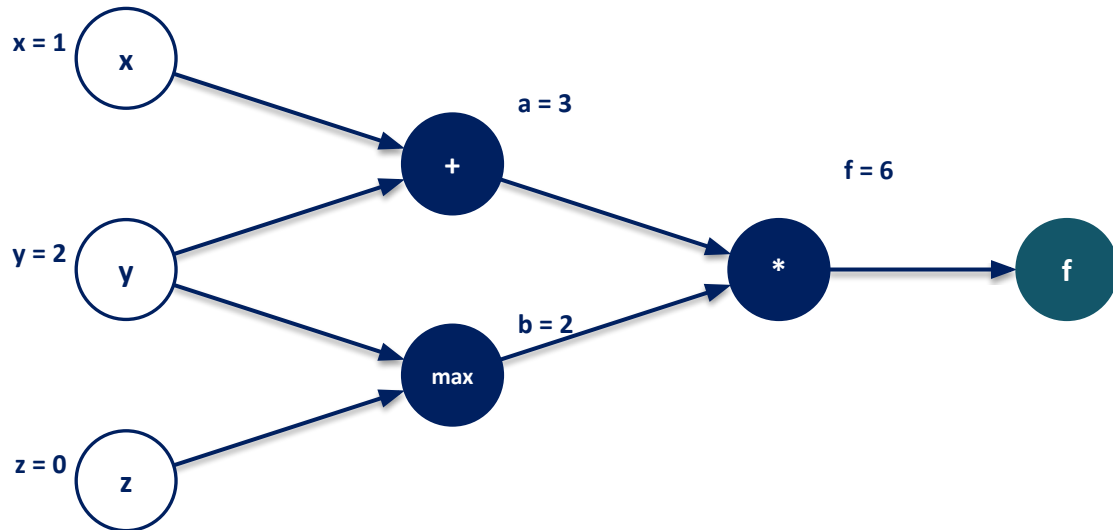
NOTE

The upstream gradient is shared

It is computed once and multiplied by each local gradient – no duplicated work.

A Worked Example (1/3): Forward Pass

$$f(x, y, z) = (x + y) \cdot \max(y, z) \quad \text{with } x = 1, y = 2, z = 0$$



three tiny operations – but the same procedure works for a billion-parameter model

FORWARD STEPS

$$a = x + y = 3$$

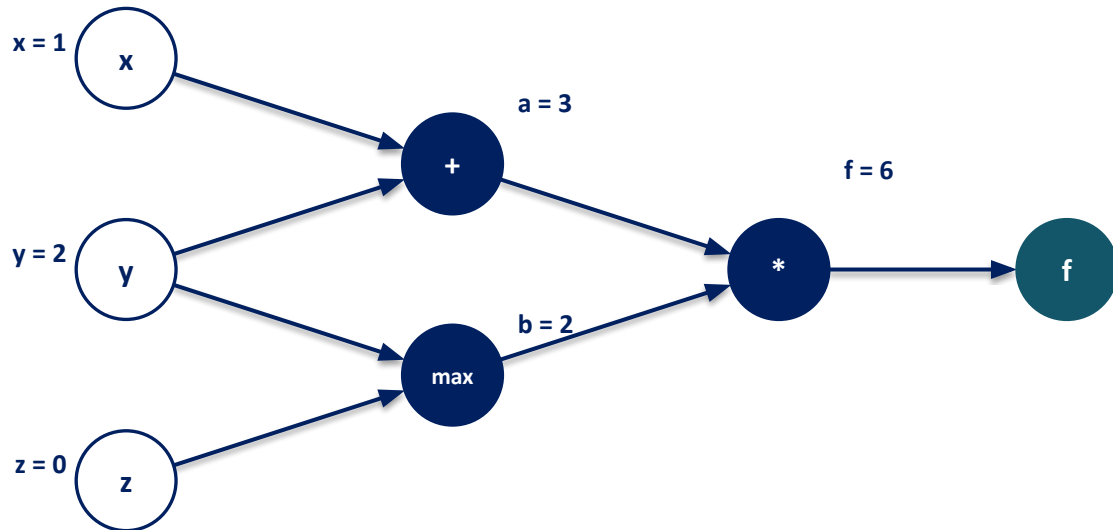
$$b = \max(y, z) = 2$$

$$f = a \cdot b = 6$$

Keep a and b – the backward pass needs them.

A Worked Example (2/3): Every Node Knows Its Local Slope

$$f(x, y, z) = (x + y) \cdot \max(y, z) \quad \text{with } x = 1, y = 2, z = 0$$



no node needs to know anything about the rest of the graph

LOCAL SLOPES

+ node ($a = x + y$)

slope 1 towards x and towards y

max node ($b = \max(y, z)$)

slope 1 towards the winner y , 0 towards z

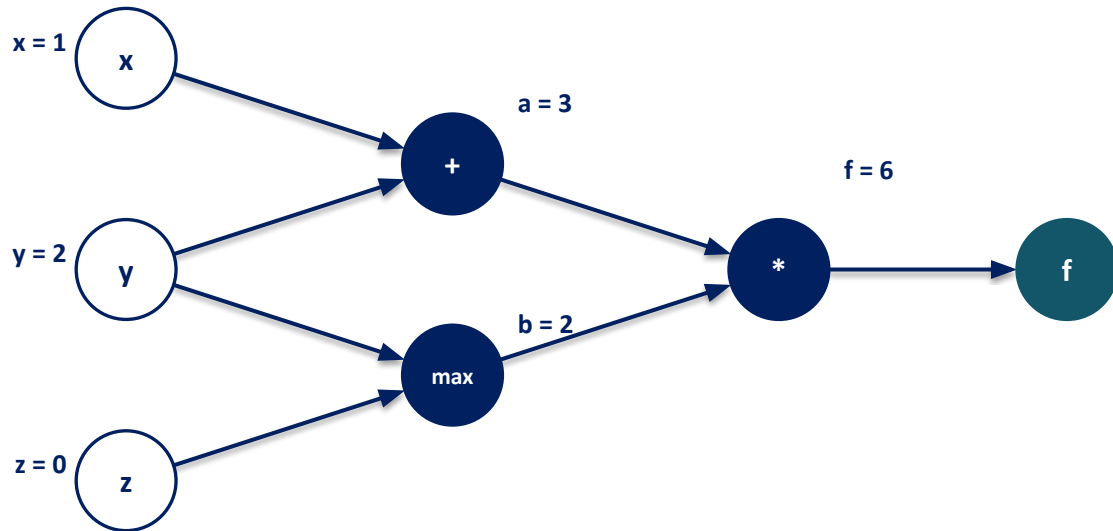
$\times$ node ($f = a \cdot b$)

slope towards a is $b = 2$; towards b is $a = 3$

Each slope uses only the saved forward values.

A Worked Example: Local Gradients at Every Node

$$f(x, y, z) = (x + y) \max(y, z) \quad x = 1, y = 2, z = 0$$



Each node only needs to know the derivative of its own output with respect to its own inputs – evaluated at the saved forward values.

LOCAL GRADIENTS

$$a = x + y: \quad \frac{\partial a}{\partial x} = 1, \quad \frac{\partial a}{\partial y} = 1$$

$$b = \max(y, z):$$

$$\frac{\partial b}{\partial y} = 1(y > z) = 1$$

$$\frac{\partial b}{\partial z} = 1(z > y) = 0$$

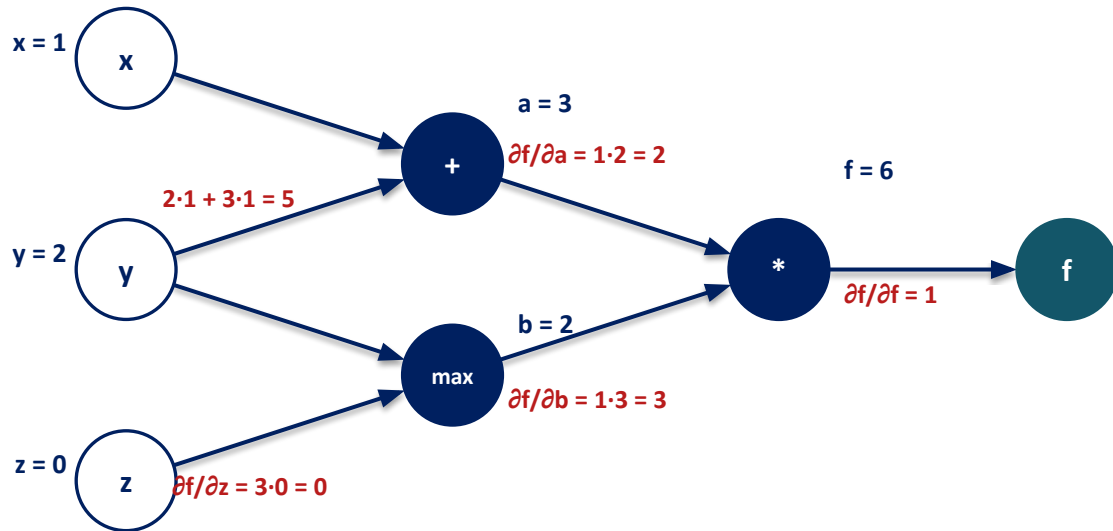
$$f = ab: \quad \frac{\partial f}{\partial a} = b = 2$$

$$\frac{\partial f}{\partial b} = a = 3$$

A Worked Example (3/3): Backward Pass

$$f(x, y, z) = (x + y) \cdot \max(y, z) \quad \text{with } x = 1, y = 2, z = 0$$

$$\partial f / \partial x = 2 \cdot 1 = 2$$



check: raise y by $0.01 \rightarrow f = 3.01 \times 2.01 = 6.05 \approx 6 + 5 \times 0.01$ ✓

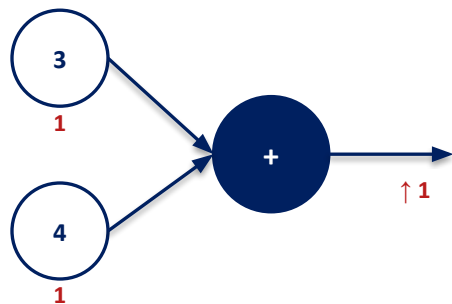
UPSTREAM $\times$ LOCAL

1. at f : gradient 1
2. to a : $1 \times 2 = 2$
3. to b : $1 \times 3 = 3$
4. to x : $2 \times 1 = 2$
5. to y : $2 \times 1 + 3 \times 1 = 5$
6. to z : $3 \times 0 = 0$

y is used twice $\rightarrow$ its two gradients add up.

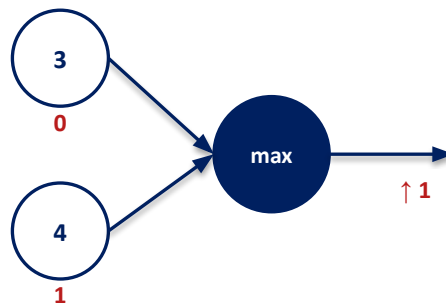
Three Node Personalities

upstream gradient = 1 arrives from the right; red numbers are what each input receives



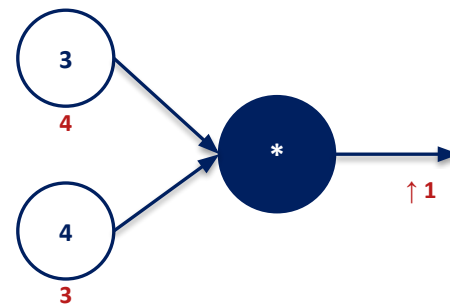
+ distributes

every input gets the upstream gradient unchanged



max routes

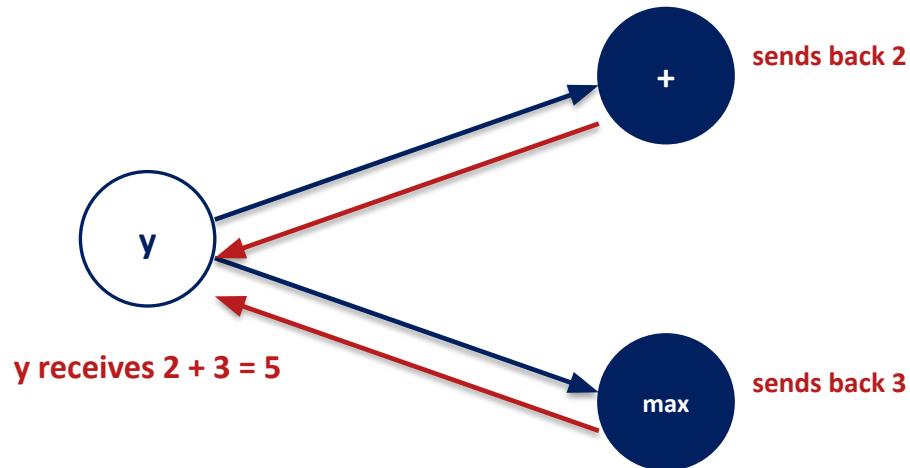
only the winning input gets the gradient; the loser gets 0



× switches

each input gets the OTHER input's value

Gradients Add Up at Branches



(forward: $a = x + y = 3$, $b = \max(y, z) = 2$)

WHY

y influences f through two paths

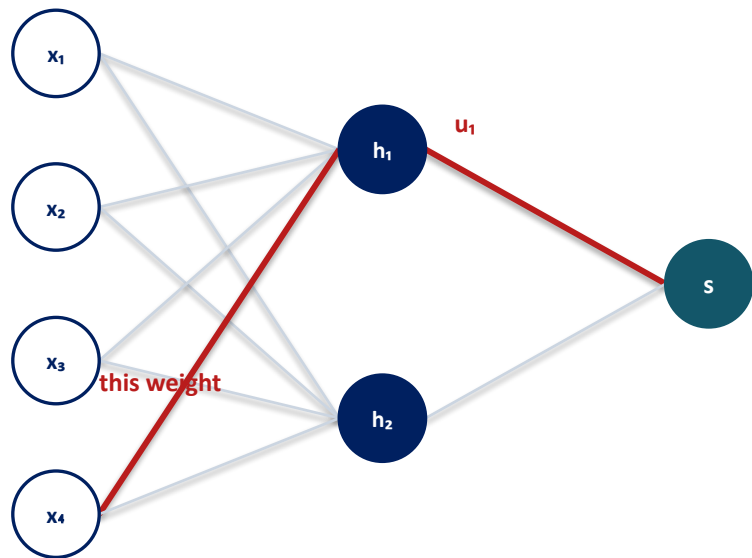
Each path contributes its own effect; total sensitivity is the sum.

WHERE THIS HAPPENS

Everywhere in real networks

A hidden unit feeds every neuron in the next layer – its gradient is a sum over all of them.

Error Sharing in a Real Network: Following One Weight



the highlighted weight only touches h_1 – so only that path matters for its gradient

THE ERROR TRAVELS BACK

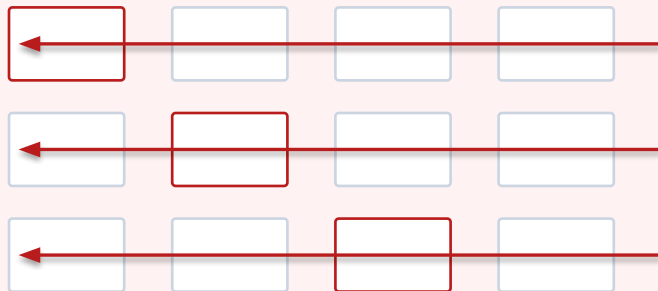
1. an error signal of 1 leaves the output s
2. crossing the edge to h_1 multiplies it by that edge's weight u_1
3. crossing the neuron multiplies it by the neuron's local slope f'
4. the weight's own gradient = (error at h_1) $\times$ (the input it carried, x_4)

Same recipe, one layer at a time, down to the word vectors – and the whole layer is done at once with matrix ops.

Why Backprop Is So Cheap

NAIVE

One walk per parameter



Three parameters $\rightarrow$ three full backward walks; the shared part of the path is recomputed each time. Millions of parameters $\rightarrow$ hopeless.

BACKPROP

One walk for everything



- Each node stores the gradient it receives and re-uses it for all its inputs
- Cost of the backward pass $\approx$ cost of the forward pass
- This is exactly the re-used δ from the by-hand derivation

Autodiff: What PyTorch Does for You

OUR WORKED EXAMPLE, IN 6 LINES

```
import torch
x = torch.tensor(1., requires_grad=True)
y = torch.tensor(2., requires_grad=True)
z = torch.tensor(0., requires_grad=True)

f = (x + y) * torch.max(y, z) # forward: 6.0
f.backward()                 # backward pass

print(x.grad, y.grad, z.grad)
# tensor(2.) tensor(5.) tensor(0.)
```

WHAT HAPPENED

Every operation recorded itself

+, max and × each appended a node to a graph as they ran; backward() walks it right-to-left.

SAME NUMBERS

2, 5, 0 – as on our slide

The framework applied exactly the rules we used by hand: distribute, route, switch, and sum at branches.

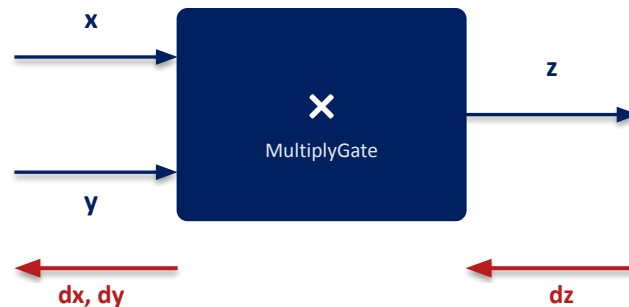
Division of labour: the framework handles the graph walk; whoever writes a new operation must still supply its local gradient.

Writing Your Own Operation: forward() and backward()

A NODE = TWO METHODS

```
class MultiplyGate:
    def forward(self, x, y):
        self.x, self.y = x, y  # save for later
        return x * y

    def backward(self, dz):
        # dz = upstream
        dx = self.y * dz        # switch!
        dy = self.x * dz
        return dx, dy          # downstream
```



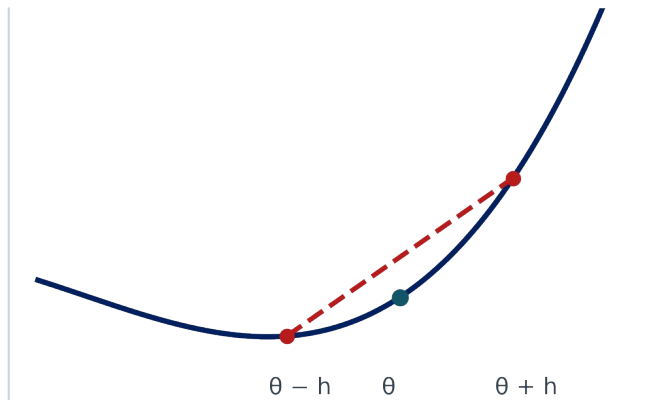
THE CONTRACT

Save what you need; multiply by upstream

forward() stores its inputs; backward() returns upstream $\times$ local slope for each input. Frameworks chain thousands of these.

Sanity Check: The Numerical Gradient

$$\text{slope} \approx (J(\theta+h) - J(\theta-h)) / 2h$$



FOUR LINES

```
def numeric_grad(J, theta, i, h=1e-4):  
    tp, tm = theta.copy(), theta.copy()  
    tp[i] += h; tm[i] -= h  
    return (J(tp) - J(tm)) / (2*h)
```

USE IT TO

Verify a hand-written backward()

Compare against the analytic gradient for a few random parameters – they should agree to $\sim 1e-6$.

WHY NOT TRAIN WITH IT?

Two forward passes per parameter

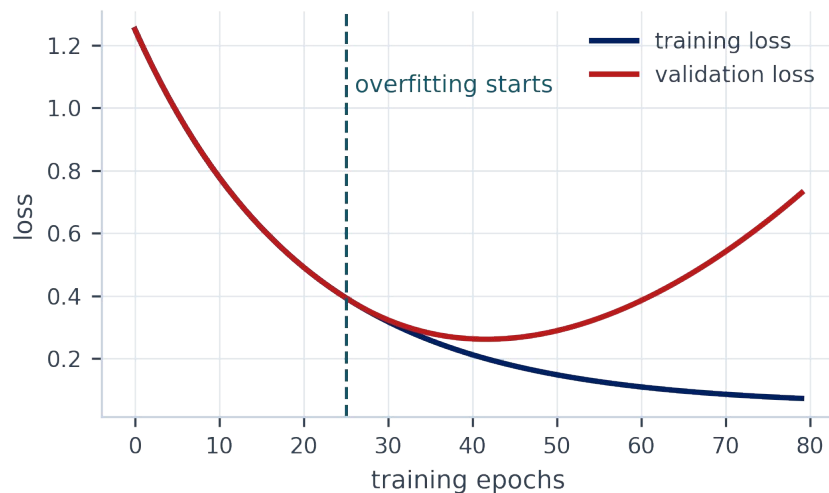
Millions of parameters $\times$ thousands of steps – far too slow. Backprop gives the same numbers in one pass. The numeric gradient is a debugging tool, not an optimizer.

4.

Making Training Work

Overfitting, dropout, initialization, optimizers – and a tiny PyTorch classifier

Overfitting & Regularization



always watch the validation curve, not the training curve

```
optim.SGD(params, lr=0.1, weight_decay=1e-4)
```

THE SYMPTOM

Training loss keeps falling, validation loss turns up

The model memorises the training set instead of learning patterns that generalise.

FIX 1 · L2 / WEIGHT DECAY

Add a penalty for large weights

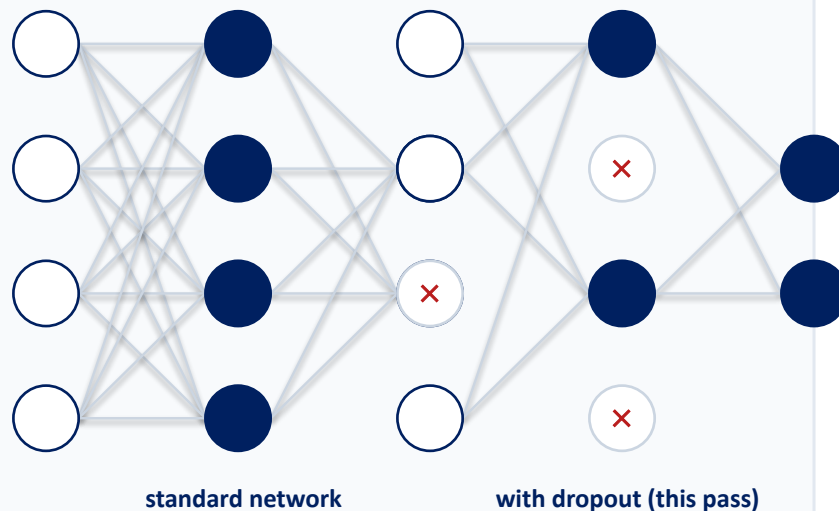
$\lambda \cdot \sum \|W\|^2$ keeps weights small → smoother, less flexible functions. Biases are not penalised.

FIX 2 · EARLY STOPPING & DROPOUT

Stop at the dashed line; randomly drop units

Next slide: dropout.

Dropout: Train Many Networks at Once



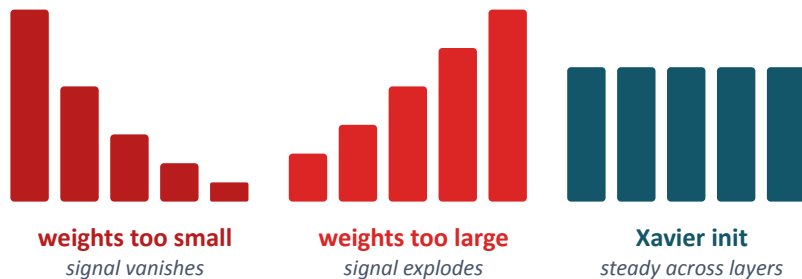
a different random subset is dropped on every pass

- **Training:** keep each hidden unit with probability p (e.g. 0.5), zero the rest; gradients flow only through survivors
- **Testing:** use the whole network, scaled so activations have the same expected size
- **Why it works:** no unit can rely on a specific partner $\rightarrow$ redundant, robust features; it is like averaging exponentially many thinner networks
- Srivastava et al., 2014 – still standard in Transformers

`nn.Dropout(p=0.5)` # one line

Initialization & Preprocessing: Start in a Good Place

size of the signal (and of gradients) as it passes through 5 layers



XAVIER / GLOROT INITIALIZATION

Scale random weights by the layer size

Uniform in $\pm\sqrt{6 / (\text{fan-in} + \text{fan-out})}$; biases start at 0. Keeps activation and gradient variance roughly constant across layers – so deep nets can train at all.

PREPROCESS THE INPUTS

Centre and scale

Subtract the training-set mean; divide by the training-set standard deviation. Apply the same numbers to validation and test data.

IN PYTORCH

Already the default

`nn.Linear` uses a Kaiming/Xavier-style init out of the box; `nn.init.xavier_uniform_(w)` if you need it explicitly.

Putting It Together: A Tiny NER Window Classifier in PyTorch

THE WHOLE SESSION IN 16 LINES

```
import torch, torch.nn as nn
d, V = 50, 10_000                # vector size, vocab
emb = nn.Embedding(V, d)          # the matrix L
model = nn.Sequential(nn.Linear(5*d, 8), # W, b
                      nn.ReLU(),        # f
                      nn.Linear(8, 1))  # u
opt = torch.optim.Adam(list(emb.parameters())
                        + list(model.parameters()), lr=1e-3)
loss_fn = nn.BCEWithLogitsLoss()

for words, label in loader:      # mini-batches
    x = emb(words).view(len(words), -1) # ① concat 5 vectors
    s = model(x).squeeze(1)           # ① forward → score
    loss = loss_fn(s, label)          # ② loss
    opt.zero_grad(); loss.backward()  # ③ backprop
    opt.step()                       # ④ update
```

① FORWARD

Embedding lookup, concatenation, Linear → ReLU → Linear. Exactly the diagram from Section 1.

② LOSS

Binary cross-entropy on the score – Section 2.

③ BACKWARD

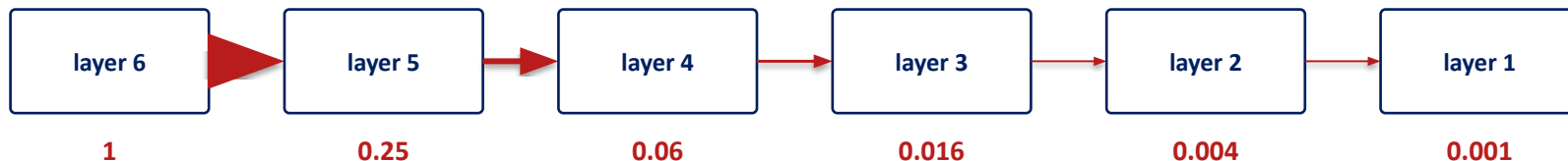
loss.backward() runs backprop through every node – Section 3. Gradients land in .grad of W, b, u and L.

④ UPDATE

Adam nudges every parameter – Section 4. zero_grad() clears the old gradients first.

Why Understand Backprop If Frameworks Do It for You?

gradient magnitude flowing back through a deep sigmoid network – each layer multiplies by a slope ≤ 0.25



← the gradient shrinks 1000× by the time it reaches the early layers: they barely learn (“vanishing gradients”)

DEBUGGING

Loss stuck? Dead ReLUs?

You can only diagnose these by reasoning about which local gradients are zero or tiny.

DESIGN

Why ReLU, residuals, LayerNorm exist

Each is a fix for how gradients flow. Next lectures: exploding/vanishing gradients in RNNs.

READ

Karpathy: “Yes, you should understand backprop”

Why take a compilers class when compilers exist? Same answer.

Summary

downstream gradient = upstream gradient $\times$ local gradient

applied backwards along the computation graph, once per node – that is backpropagation

① NEURAL NETS

**Layers of weighted sums
+ non-linearities**

Word vectors in, scores out;
every layer re-represents its
input.

② TRAINING

**Forward $\rightarrow$ loss $\rightarrow$
backward $\rightarrow$ update**

Gradient descent with a
tuned learning rate, on
mini-batches.

③ BACKPROP

**One backward walk gives
every gradient**

+ distributes, max routes, $\times$
switches, branches add up.

④ PRACTICE

**Regularize, initialize,
optimize**

Dropout, weight decay, Xavier
init, Adam – and PyTorch does
the bookkeeping.

Advanced NLP - Session 3 - Neural Network Basics & Backpropagation in NLP - IUST

Thank you! Questions?

REFERENCES

Diyi Yang (2025). CS224N Lecture 3: Neural net learning – Gradients by hand and algorithmically (backpropagation). Stanford University.

Mundra, Peddada, Socher, Yan (2019). CS224n Lecture Notes Part III: Neural Networks, Backpropagation. Stanford University.

Karpathy (2016). Yes, you should understand backprop. · Srivastava et al. (2014). Dropout.

Course site: <https://web.stanford.edu/class/cs224n/>